Project Report: Phishing URL and Email Detection

Program: IBM SkillsBuild Cybersecurity Bootcamp 2026

Organization: BharatCares

Student name: KAMINI SHANMUKHA

Institute: IDEAL INSTITUTE OF TECHNOLOGY KAKINADA

Submission date: 22-10-2026

1. Abstract

Phishing is a social-engineering technique that attempts to trick people into disclosing credentials, making payments, or opening harmful content. This project implements a Python command-line prototype that checks URLs and email text for selected suspicious indicators. It applies explainable rules and returns a Low, Medium, or High heuristic risk level with reasons. The system is intended for cybersecurity education and awareness, not as a replacement for enterprise security controls.

2. Problem statement

Users may have difficulty recognizing deceptive links and manipulative email wording. A simple, transparent tool can help learners understand common warning signs before interacting with potentially harmful content.

3. Objectives

• Build a Python tool that accepts a URL or email text.
• Identify common URL and email warning indicators.
• Provide a risk band and human-readable explanations.
• Demonstrate defensive cybersecurity programming using safe, fictional examples.
• Document limitations and responsible use.

4. Scope

The prototype analyzes user-supplied text locally and does not open URLs or download content. It checks a limited set of URL structure features and email phrases. It does not determine true domain reputation, inspect attachments, validate SPF/DKIM/DMARC, or use a machine-learning model.

5. Tools and technologies

Python 3.10 or newer; Python standard library modules `re`, `ipaddress`, and `urllib.parse`; a text editor or IDE; Git and GitHub for version control and submission.

6. Methodology

1. **Input:** User enters a URL or pastes email text into the CLI.
2. **Parsing:** URL text is parsed to inspect scheme, hostname, and authority.
3. **Rule checks:** Checks include non-HTTPS scheme, IP host, long URL, user-info, punycode labels, many subdomain levels, URL shorteners, urgency terms, and selected TLDs. Email checks include urgency, credential requests, financial requests, click/download language, and extracted URLs.
4. **Scoring:** Configured indicators add points mapped to Low, Medium, or High risk.
5. **Explanation:** Detected indicators are displayed.

7. Implementation

The application is implemented in `app.py`. `analyze_url()` evaluates a URL; `extract_urls()` extracts URL-like strings; and `analyze_email()` evaluates email wording and analyzes extracted links. The interface is an interactive command-line menu.

8. Testing plan and sample cases

Use fictional examples only and do not visit suspicious links.

T1: `https://example.com/` — no configured indicators or low score.
T2: `http://198.51.100.25/verify-account?urgent=1` — flags non-HTTPS, IP host, and suspicious wording.
T3: Urgent account email containing the fictional URL — flags urgency/account-verification wording and URL indicators.
T4: Empty URL/email — returns Unknown.

T5: URL with @ in authority — flags potentially confusing authority syntax.

Run the program and tests, then record actual results before final submission.

9. Results and discussion

The prototype demonstrates a transparent rule-based approach and produces human-readable explanations. It can support classroom demonstrations of common phishing signals. It has not been evaluated on a representative labeled dataset, so no accuracy, precision, recall, or F1-score is claimed. The score is a heuristic indicator, not a calibrated probability.

10. Limitations

• Heuristics may flag legitimate links or miss sophisticated phishing attempts.
• HTTPS does not establish trust.
• Keyword and TLD checks are context-sensitive and may be inaccurate.
• Sender identity, headers, attachments, redirects, page content, and live threat intelligence are not checked.
• Not suitable as the sole basis for security decisions.

11. Ethical and safety considerations

The project is intended for defensive education. It does not send emails, exploit systems, or visit URLs. Use fictional examples or content you are authorized to analyze. Avoid entering confidential data, passwords, OTPs, or private emails into shared systems. If a message appears suspicious, do not click its links; report it through the appropriate organizational process.

12. Future enhancements

• Evaluate against a responsibly sourced labeled dataset.
• Add automated tests and measurable metrics.
• Add email header authentication checks.
• Add optional, privacy-conscious threat-intelligence lookups.
• Build a simple web interface with clear warnings and safe handling of input.

13. Conclusion

This project provides a beginner-friendly Python demonstration of phishing awareness using interpretable URL and email heuristics. It identifies selected suspicious indicators while communicating that automated scores are imperfect. Additional testing and validation are required before real-world deployment.

14. References

Python documentation: https://docs.python.org/3/
CISA, Recognize and Report Phishing: https://www.cisa.gov/secure-our-world/recognize-and-report-phishing
Google Safe Browsing: https://safebrowsing.google.com/ (background reference; not integrated in this prototype).

Before submitting: Run the code and tests, and follow any official bootcamp format instructions.